

ЛАБОРАТОРНАЯ РАБОТА №11

РАЗРАБОТКА ФУНКЦИОНАЛЬНОСТИ ПРИЛОЖЕНИЯ HANGMAN

Что делаем сегодня?

На прошлой ЛР10 мы сделали **интерфейс** нашего приложения “Виселица” – рисовали стойку и человечка, а также кодом создавали клавиатуру,

А в этой мы напишем саму **логику** – правила, выведем слово, и тд.

Интересно: сегодня мы не создадим в XML **ни одной кнопки** – все 32 клавиши появятся из кода циклом. К концу пары поймёте, почему это лучше, чем ~200 строк XML.

Основные правила

1. Лабораторная работа разделена на **ЧАСТИ** – выполняете каждую из частей и **показываете**.
2. Обычно в конце каждой части есть **ЗАДАНИЕ** – его надо сделать
3. Не бойтесь **задавать любые** вопросы
4. Если хотите сделать что-то **свое**, то **предлагайте** – будем думать
5. Советую читать текст, тут **написано** много полезного
- 6.



Часть 1. Делаем состояние игры

В прошлой ЛР в **HangmanView** был временный метод, который по нажатию менял этап (то есть рисовал человечка)- сейчас у этапов появляется настоящий источник, и он больше не нужен:

```
// ВРЕМЕННО: нажатие переключает этап
@Override
public boolean onTouchEvent(MotionEvent event) {
    if (event.getAction() == MotionEvent.ACTION_DOWN) {
        stage++; // меняем этап на 1 больше
        postInvalidate();

        Log.d( tag: "Hangman", msg: "Этап: " + stage);
        return true;
    }
    return super.onTouchEvent(event);
}
```

Откройте **HangmanView** и удалите весь метод **onTouchEvent**. Все остальное мы оставляем.

Теперь сделаем так, чтобы хранилось **состояние нашей игры** и **словарь** наших слов (можете добавить свои слова, в том числе), ТАКЖЕ не забываем **найти элементы с экрана** по их id и, заходим в **MainActivity**:

```
24 <> public class MainActivity extends AppCompatActivity {
25
26     // СЛОВАРЬ СЛОВ ДЛЯ ИГРЫ
27     // все слова ЗАГЛАВНЫМИ - потому что клавиатура тоже заглавными,
28     private static final String[] WORDS = { 2 usages
29         "ПРОГРАММА", "КОМПЬЮТЕР", "АЛГОРИТМ", "СВЯЗЬ", "КЛАВИАТУРА",
30         "ЭМУЛЯТОР", "МОНИТОР", "СТУДЕНТ", "ОМАВИАТ", "СВЯЗЬ"
31     };
32
33     // состояние игры
34     private String secretWord; // загаданное слово 9 usages
35     private StringBuilder mask; // какие буквы открыты: "ПР_ГР_ММ_" 6 usages
36     private int mistakes; // количество ошибок 6 usages
37     private boolean gameEnded; // окончена ли игра? 3 usages
38     private final Random random = new Random(); 1 usage
39
40     // элементы экрана
41     private HangmanView hangmanView; 3 usages
42     private TextView tvMask; 3 usages
43     private TextView tvStatus; 3 usages
44     private final List<Button> allButtons = new ArrayList<>(); 2 usages
45
46     @Override
47     protected void onCreate(Bundle savedInstanceState) {
48         super.onCreate(savedInstanceState);
49         EdgeToEdge.enable(this);
50         setContentView(R.layout.activity_main);
51         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (View v, WindowInsetsCompat insets) -> {
52             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
53             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
54             return insets;
55         });
56
57         // находим элементы
58         hangmanView = findViewById(R.id.hangmanView);
59         tvMask = findViewById(R.id.tvMask);
60         tvStatus = findViewById(R.id.tvStatus);
61
62         buildKeyboard();
63     }
```

Вы увидели в коде **StringBuilder** – это “изменяемая строка”. Обычную String в Java нельзя изменить после создания, любая правка создаёт новую строку, например:

```
String str = "Hello, ";
str += "World!";
System.out.println(str);
```

В этом примере может показаться, что мы **изменили str**, но на самом деле произошло то, что **новый объект String** был создан с **комбинированным содержимым**.

Исходный строковый объект “Hello, ” **все еще существует** в памяти, но он потерян для нас, потому на него больше нет ссылки. А теперь то же самое при помощи **StringBuilder**:

```
StringBuilder sb = new StringBuilder("Hello, ");
sb.append("World!");
System.out.println(sb.toString());
```

Теперь добавим в **MainActivity** метод для запуска новой игры и метод который будет создавать красивый вывод загаданного слова:

```
private void startNewGame() {
    // выбираем случайное слово из нашего словаря
    secretWord = WORDS[random.nextInt(WORDS.length)];

    // создаем МАСКУ той же длины, что слово - пока что все буквы пропуски
    mask = new StringBuilder();
    for (int i = 0; i < secretWord.length(); i++) {
        mask.append('_');
    }

    // обнуляем состояние нашей игры
    mistakes = 0;
    gameEnded = false;
    hangmanView.setStage(0);
    // выводим состояние
    tvMask.setText(formatMask());
    tvStatus.setText("Осталось попыток: " + HangmanView.MAX_STAGES);

    Log.d("tag: Hangman", "msg: Новое слово: " + secretWord);
}

// делаем МАСКУ с пробелами между буквами: "_ _ _ _ _"
private String formatMask() {
    StringBuilder pretty = new StringBuilder();
    for (int i = 0; i < mask.length(); i++) {
        if (i > 0) pretty.append(' ');
        pretty.append(mask.charAt(i));
    }
    return pretty.toString();
}
```

Метод **buildKeyboard()** и всё остальное из ЛР10 пока оставьте как было – кнопки все еще просто логируют нажатия. Не забываем дополнить и **onCreate**:

```
// находим элементы
hangmanView = findViewById(R.id.hangmanView);
tvMask = findViewById(R.id.tvMask);
tvStatus = findViewById(R.id.tvStatus);

buildKeyboard();
findViewById(R.id.btnNewGame).setOnClickListener( View v -> startNewGame());

startNewGame();
}
```

! ЗАДАНИЕ !

Запустите и несколько раз нажмите **“Новая игра”** – в **Logcat** (фильтр tag:Hangman) каждое нажатие пишет новое слово, маска на экране меняет длину под него.

Часть 2. Делаем проверку нажатий

Замените старый **onLetterPressed** на новый:

```
private void onLetterPressed(Button button) { 1 usage
    //проверяем - закончилась ли игра
    if (gameEnded) return;
    // получаем букву из кнопки
    char letter = button.getText().charAt(0);

    // проверяем - есть ли буква в загаданном слове
    if (secretWord.indexOf(letter) >= 0) {
        button.setBackgroundColor(Color.GREEN);
        // открываем эту букву в слове
        for (int i = 0; i < secretWord.length(); i++) {
            if (secretWord.charAt(i) == letter) {
                mask.setCharAt(i, letter);
            }
        }
        tvMask.setText(formatMask());
    }
    // если буквы нет - идем внутрь else
} else {
    button.setBackgroundColor(Color.RED);
    mistakes++;
    hangmanView.setStage(mistakes);
    tvStatus.setText("Осталось попыток: " + (HangmanView.MAX_STAGES - mistakes));
}
}
```

Важная деталь:

Почему проходимся в цикле по слову? – например, в слове ПРОГРАММА буква Р встречается дважды, и М тоже, и А – игрок угадал букву, без цикла мы бы открыли одну из трёх.

Поэтому проходим по всему слову и открываем каждое совпадение.

Давайте попробуем запустить и поиграть. Загляните в **Logcat** – там написано загаданное слово. **Отгадайте 2–3 буквы, потом заведомо отсутствующие:** маска открывается, человечек дорисовывается, счётчик уменьшается.

Однако у нас есть ошибка. **Нажмите одну и ту же отсутствующую букву два раза:** ошибки станут две, а на виселице прибавится два элемента – хотя игрок нажал одну букву.

Но одну и ту же букву **ДВАЖДЫ**. Кнопка-то не блокируется, и каждое нажатие считается.

! ЗАДАНИЕ !

Сделайте чтобы кнопка после нажатия блокировалась. У объектов типа **Button** есть метод – **setEnabled(false)** – в который можно закинуть false и кнопка выключится.

Часть 3. Победа и поражение

Осталось определить конец игры и красиво его показать. Условия:

- **победа** – это когда в нашей маске **не осталось “_”**
- **поражение** – ошибки достигли **MAX_STAGES**.

Добавьте проверки в конец **onLetterPressed**:

```
// если пропусков больше нет - мы угадали все буквы
if (mask.indexOf("_") == -1) {
    endGame( won: true);
}

// проверяем - есть ли у нас еще попытки?
if (mistakes >= HangmanView.MAX_STAGES) {
    endGame( won: false);
}
```

Обратите внимание что:

- если **mask.indexOf("_")** не найдет элемент, то он вернёт -1

А теперь создадим сам **метод конца игры**:

```
private void endGame(boolean won) { 2 usages
    gameEnded = true;

    // если выиграли показываем сообщение о победе
    if (won) {
        Log.i( tag: "Hangman", msg: "Победа!");
        new AlertDialog.Builder( context: this)
            .setTitle("Победа!")
            .setMessage("Слово " + secretWord + " отгадано! Ошибок: " + mistakes)
            .setPositiveButton( text: "Новая игра", ( DialogInterface d, int w) -> startNewGame())
            .setNegativeButton( text: "Отмена", listener: null)
            .show();
    }
    // иначе сообщение о проигрыше
    else {
        Log.i( tag: "Hangman", msg: "Проигрыш, слово: " + secretWord);
        new AlertDialog.Builder( context: this)
            .setTitle("Проигрыш")
            .setMessage("Слово было: " + secretWord)
            .setPositiveButton( text: "Новая игра", ( DialogInterface d, int w) -> startNewGame())
            .setNegativeButton( text: "Отмена", listener: null)
            .show();
    }
}
```

Новое здесь:

AlertDialog – окно поверх экрана с заголовком, текстом и кнопками.

! ЗАДАНИЕ !

У нас есть кнопка “Новая игра”, но клавиши после проигрыша остались серыми и выключенными.

Добавьте в конец **startNewGame()** разблокировку (именно для этого мы ранее создали список **allButtons**, куда мы сохраняли кнопки). Нужно пройти по всем кнопкам в списке и включить их (также не забудьте исправить цвет)

Требования к итоговой версии

1. Случайное слово из словаря не менее чем из 10 слов
2. Маска и счётчик **обновляются**
3. Угаданная буква открывается во всех своих вхождениях
4. Ошибка **дорисовывает** человечка, а счётчик попыток **уменьшается**
5. **Повторное нажатие** и **нажатие после конца игры** игнорируются
6. “Новая игра” полностью **сбрасывает игру**
7. Существенные события пишутся в **Logcat**

Дополнительно (по желанию)

1. **Категории.** Два-три словаря (животные, города, IT или что-то еще), случайная категория на игру, её название отобразите – в статусе. (Подсказка: String[] или отдельные массивы + случайный индекс)
2. **Счёт.** Добавьте поля wins/losses, и показывайте их число в статусе и обновляйте в endGame.
3. **Сдаюсь.** Сделайте кнопку, которая открывает всё слово в маске и вызывает endGame(false) без шести ошибок.
4. **HashMap.** Используйте для сохранения кнопок вместо **ArrayList** - **HashMap** (словарь “ключ-значение”), где в виде ключа будет выступать буква, а значением кнопка.